

Khatri Yashvi

(202351067)

Assignment-9

Distributed Bank Backend

Objective

To design and implement a distributed banking system using MPI, where two independent nodes compute compound interest with random market variations and simulated audit delay, exchange results through inter-process communication, and verify the final total using a consensus mechanism.

Code

```
from mpi4py import MPI
import random
import time
import sys

def compute_maturity(principal, years, delay_ms, seed):
    random.seed(seed)

    amount = float(principal)
    base_rate = 0.07
    total_days = years * 365

    for _ in range(total_days):
        bonus = random.uniform(0.001, 0.009)
        daily_rate = (base_rate + bonus) / 365.0
        amount = amount * (1 + daily_rate)
```

```

        time.sleep(delay_ms / 1000.0)

    return amount

def main():
    comm = MPI.COMM_WORLD
    rank = comm.Get_rank()
    size = comm.Get_size()

    if size != 2:
        if rank == 0:
            print("Error: Run this program with exactly 2 processes.")
            print("Example: mpiexec -n 2 python distributed_bank.py")
            sys.exit()

    # Take input only from Node A / Rank 0
    if rank == 0:
        P1 = int(input("Enter Client 1 principal (last 4 digits): "))
        P2 = int(input("Enter Client 2 principal (last 4 digits): "))
        years = int(input("Enter number of years: "))
        delay_ms = int(input("Enter audit delay per day in
milliseconds: "))
    else:
        P1 = None
        P2 = None
        years = None
        delay_ms = None

    # Broadcast inputs to both nodes
    P1 = comm.bcast(P1, root=0)
    P2 = comm.bcast(P2, root=0)
    years = comm.bcast(years, root=0)
    delay_ms = comm.bcast(delay_ms, root=0)

    # Assign node data
    if rank == 0:
        node_name = "Node A"
        my_principal = P1
        my_seed = P1
    else:
        node_name = "Node B"
        my_principal = P2

```

```

    my_seed = P2

start_time = MPI.Wtime()

# Phase 1 + Phase 2
my_maturity = compute_maturity(my_principal, years, delay_ms,
my_seed)

# Phase 3: Message passing
other_rank = 1 - rank
received_maturity = comm.sendrecv(
    sendobj=my_maturity,
    dest=other_rank,
    source=other_rank
)

# Phase 4: Consensus
final_total = my_maturity + received_maturity
other_total = comm.sendrecv(
    sendobj=final_total,
    dest=other_rank,
    source=other_rank
)

end_time = MPI.Wtime()

print(f"{node_name} (Rank {rank}) Report")
print(f"Principal Processed : {my_principal}")
print(f"Random Seed          : {my_seed}")
print(f"Years Processed        : {years}")
print(f"Audit Delay / Day      : {delay_ms} ms")
print(f"Final Maturity          : {my_maturity:.6f}")
print(f"Received Other Value: {received_maturity:.6f}")
print(f"Final Total             : {final_total:.6f}")

if abs(final_total - other_total) < 1e-9:
    print("Consensus Status      : SUCCESS (Both nodes matched)")
else:
    print("Consensus Status      : FAILED (Mismatch detected)")

print(f"Execution Time          : {end_time - start_time:.6f} seconds")

```

```
if __name__ == "__main__":  
    main()
```

Output

```
mpiexec -n 2 python distributed_bank.py  
Enter Client 1 principal (last 4 digits): 3934  
Enter Client 2 principal (last 4 digits): 3453  
Enter number of years: 3  
Enter audit delay per day in milliseconds: 4
```

Node A (Rank 0) Report

```
Principal Processed : 3934  
Random Seed       : 3934  
Years Processed   : 3  
Audit Delay / Day : 4 ms  
Final Maturity    : 4926.875853  
Received Other Value: 4324.525595  
Final Total       : 9251.401447  
Consensus Status  : SUCCESS (Both nodes matched)  
Execution Time    : 5.919906 seconds
```

Node B (Rank 1) Report

```
Principal Processed : 3453  
Random Seed       : 3453  
Years Processed   : 3  
Audit Delay / Day : 4 ms  
Final Maturity    : 4324.525595  
Received Other Value: 4926.875853  
Final Total       : 9251.401447  
Consensus Status  : SUCCESS (Both nodes matched)  
Execution Time    : 5.921992 seconds
```
